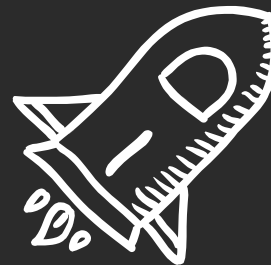




Semana 15 ¡Bienvenidos!



Temas de hoy

1

Deploy de proyecto en Python Anywhere

1

Deploy de Proyecto

Es momento de subir nuestro proyecto a producción. Ya construimos, configuramos nuestro proyecto ahora tenemos que mostrar y publicar lo que creamos.

Si bien hay muchos proveedores de servidores disponibles en Internet. Nosotros vamos a utilizar uno que tiene un proceso relativamente sencillo: **PythonAnywhere** y necesitaremos tener nuestro repositorio en Github creado. 😊

Deploy de Proyecto: Pasos Previos

Repositorio en Github

Antes de detallar los pasos para subir nuestro proyecto online es muy importante que tengamos creado y configurado nuestro repositorio en Github.

Por si no recordas los comandos para crear y subir los archivos a tu repositorio te dejamos a continuación una lista de comandos para que puedas consultar:

https://training.github.com/downloads/es_ES/github-git-heatmap-sheet.pdf

Creación de cuenta en PythonAnywhere

Ahora que ya tenemos nuestro repositorio creado, tenemos que crear una cuenta gratuita (Beginner Account) en PythonAnywhere:

<https://www.pythonanywhere.com/pricing/>

→ Cuando te hayas registrado en PythonAnywhere te solicitará que confirmes tu correo electrónico:



Confirm your email address

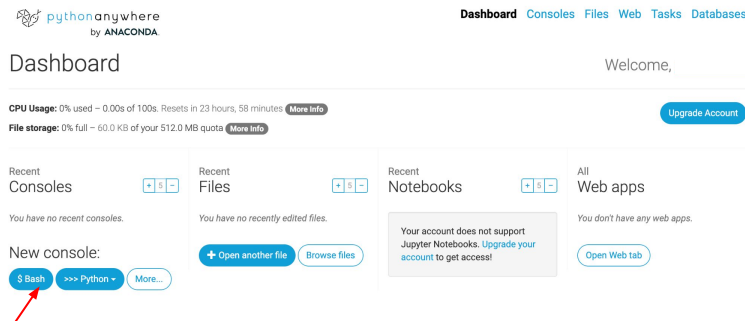
All users now need to have a confirmed email address to use PythonAnywhere. Check your inbox for an email from us and click the link to continue. If you can't find the email, request a new one with the button below.

Resend email

Deploy de Proyecto: Configuración

Cuando te hayas registrado en PythonAnywhere serás redirigido a tu panel de control o página "Consoles".

Elige la opción para iniciar una consola "Bash". Esta es la versión PythonAnywhere de una consola, como la que tienes en tu PC.



Vamos a descargar nuestro código de GitHub a PythonAnywhere mediante la creación de un "clon" del repositorio.

Escribe lo siguiente en la consola de PythonAnywhere (no te olvides de utilizar tu nombre de usuario de GitHub en lugar de <tu-usuario-github>):



Bash console 36119789

```
22:45 ~ $ git clone https://github.com/natalibogarin/proyectoblogdjango.git
Cloning into 'proyectoblogdjango'...
remote: Enumerating objects: 362, done.
remote: Counting objects: 100% (362/362), done.
remote: Compressing objects: 100% (249/249), done.
remote: Total 362 (delta 147), reused 305 (delta 95), pack-reused 0 (from 0)
Receiving objects: 100% (362/362), 1.87 MiB | 13.66 MiB/s, done.
Resolving deltas: 100% (147/147), done.
Updating files: 100% (107/107), done.
22:46 ~ $
```

Con el comando `git clone` <link repositorio> vas a copiar el código de tu repositorio en PythonAnywhere.

Compruébalo escribiendo `tree` <nombre carpeta>:

```
22:46 ~ $ tree proyectoblogdjango
proyectoblogdjango
├── Procfile
├── apps
│   ├── blog_auth
│   │   ├── __init__.py
│   │   ├── pycache__
│   │   └── __init__.cpython-310.pyc
```

Deploy de Proyecto: Configuración

Tal y como lo hicimos en nuestra computadora, el próximo paso es crear un virtualenv en PythonAnywhere. En la consola Bash, escribe:

```
cd <nombr carpetarepositorio>  
virtualenv --python=python3.10 <nombreentorno>
```

Luego debes activar el entorno escribiendo:
source nombreentorno/bin/activate

 Bash console 36119789

```
22:55 ~ $ cd proyectoblogdjango/  
22:55 ~/proyectoblogdjango (main)$ virtualenv --python=python3.10 mientorno  
created virtual environment CPython3.10.5.final.0-64 in 16074ms  
creator CPython3Posix(dest=/home/nataliab77/proyectoblogdjango/mientorno, clear=False, no_vcs_ignore=False,  
seeder FromAppData(download=False, pip=bundle, setuptools=bundle, wheel=bundle, via=copy, app_data_dir=/home/  
added seed packages: pip=22.1.2, setuptools=62.6.0, wheel=0.37.1  
activators BashActivator,CShellActivator,FishActivator,NushellActivator,PowerShellActivator,PythonActivator  
22:56 ~/proyectoblogdjango (main)$ source mientorno/bin/activate  
(mientorno) 22:56 ~/proyectoblogdjango (main)$
```

Una vez activado el entorno, hay que instalar Django y las dependencias que utilizamos en nuestro proyecto:

```
pip install -r /path/to/requirements.txt
```

```
(mientorno) 23:00 ~/proyectoblogdjango (main)$ pip install -r requirements.txt  
Looking in links: /usr/share/pip-wheels  
Collecting asgiref==3.5.2  
  Downloading asgiref-3.5.2-py3-none-any.whl (22 kB)  
Collecting autopep8==1.7.0  
  Downloading autopep8-1.7.0-py2.py3-none-any.whl (45 kB)  
    45.6/45.6 kB 807.6 kB/s eta 0:00:00  
Collecting distlib==0.3.5  
  Downloading distlib-0.3.5-py2.py3-none-any.whl (466 kB)  
    467.0/467.0 kB 9.3 MB/s eta 0:00:00  
Collecting dj-database-url==1.0.0  
  Downloading dj_database_url-1.0.0-py3-none-any.whl (6.6 kB)  
Collecting Django==4.0.6  
  Downloading Django-4.0.6-py3-none-any.whl (8.0 MB)  
    8.0/8.0 MB 25.4 MB/s eta 0:00:00
```

Deploy de Proyecto: Configuración

El próximo paso es la configuración de la base de datos. Tienes que actualizar en el archivo settings.py el motor de la base de datos, en este caso la cuenta gratuita solo permite utilizar sqlite3.

De la misma forma que hicimos en nuestra computadora, repetimos los pasos para inicializar la base de datos en el servidor, con migrate:

```
(mientorno) 23:09 ~/proyectoblogdjango (main)$ python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, blog_auth, contenttypes, noticias_app, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying blog_auth.0001_initial... OK
  Applying noticias_app.0001_initial... OK
```

Creamos nuestro usuario de acceso al Panel de Administración utilizando createsuperuser:

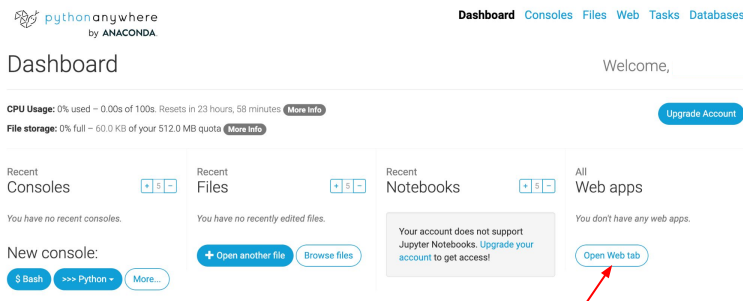
```
(mientorno) 23:11 ~/proyectoblogdjango (main)$ python manage.py createsuperuser
Nombre de usuario (leave blank to use 'nataliabogarin'): nataliabogarin
Dirección de correo electrónico: natali.bogarin@gmail.com
Password:
Password (again):
Esta contraseña es completamente numérica.
Bypass password validation and create user anyway? [y/N]: y
Superuser created successfully.
```

Ahora nuestro código está en PythonAnywhere, el virtualenv está listo y la base de datos está inicializada.

¡Está todo listo para publicar nuestra aplicación web!

Deploy de Proyecto: Configuración

Volvemos al Panel de Control de Python Anywhere, haciendo clic en el logo de PythonAnywhere. Por último, seleccionamos en Open Web Tab:



Seleccionamos Add a new web app y luego hacemos clic en siguiente. Seleccionamos "Manual configuration" y Django 3.10 como versión de Python.

Luego configuramos la ruta de acceso a nuestro proyecto:

Code:

What your site is running.

Source code: </home/nataliab77/proyectoblogdjango>

[Go to directory](#)

Working directory: </home/nataliab77/proyectoblogdjango>

[Go to directory](#)

WSGI configuration file: </var/www/nataliab77.pythonanywhere.com/wsgi.py>

Python version: 3.10 

Configuramos el entorno virtual y la ruta a los archivos estáticos:

Virtualenv:

Use a virtualenv to get different versions of flask, django etc from our default system ones. [More info here](#). You need to **Reload your web app** to activate it; NB - will do nothing if the virtualenv does not exist.

</home/nataliab77/proyectoblogdjango/mientorno>

[Start a console in this virtualenv](#)

Deploy de Proyecto: Configuración

Por último tenemos que configurar el archivo `wsgi.py` agregando el siguiente código:

```
import os
import sys

path = os.path.expanduser('~/.projectoblogdjango')
if path not in sys.path:
    sys.path.append(path)

os.environ['DJANGO_SETTINGS_MODULE'] = 'projectoblogdjango.base'

from django.core.wsgi import get_wsgi_application
from django.contrib.staticfiles.handlers import StaticFilesHandler
application = StaticFilesHandler(get_wsgi_application())
```

En el caso de que ocurra algún error, podemos consultar los archivos de logs:

Something went wrong :-)

Something went wrong while trying to load this website; please try again later.

If it is your site, you should check your logs to determine what the problem is.

There was an error loading your PythonAnywhere-hosted site. There may be a bug in your code.

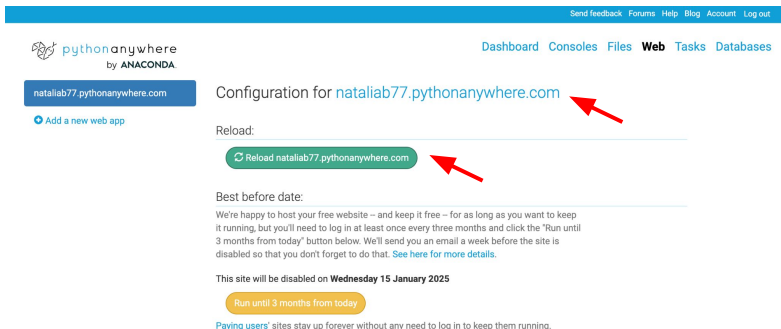
Error code: Unhandled Exception

Debugging tips

- The first place to look is at your [web app page](#) to ensure that there are no errors indicated there.
- Next, check your site's server and error logs for any messages — you can view them here:
 - [nataliab77.pythonanywhere.com.error.log](#)
 - [nataliab77.pythonanywhere.com.server.log](#)
- You can find helpful tips on the PythonAnywhere help site:
 - [There's an ImportError in the logs](#)
 - ["403 Forbidden" error or "Connection Refused" error in logs](#)
 - [Database connection errors](#)

Deploy de Proyecto

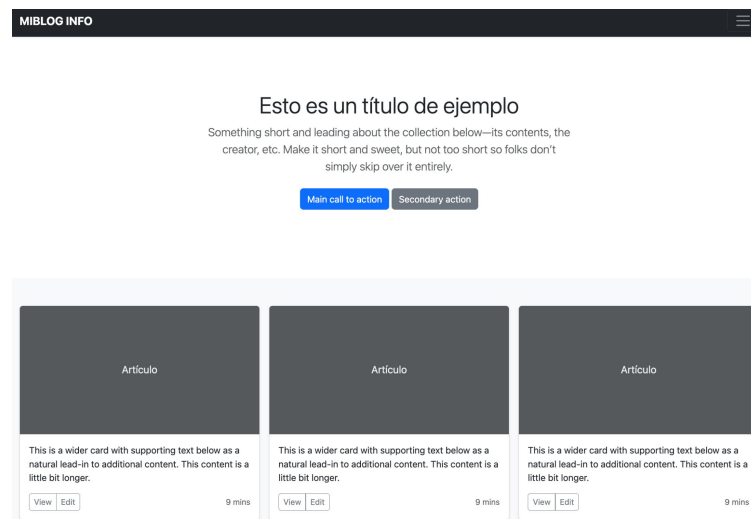
Una vez que culminamos la configuración volvemos al Panel principal:



La URL que figura en el título es donde está publicada nuestra WebApp.

En el botón verde podemos recargar la aplicación en el caso que hayamos tenido errores previos.

Cuando seleccionemos la URL veremos nuestra aplicación online: 🙌🙌





Conclusión final

Realizar el deploy de una web app en Python usando PythonAnywhere es un proceso accesible que permite a los desarrolladores desplegar aplicaciones de manera rápida y sencilla en un entorno de producción.

El enfoque provisto por esta herramienta nos proporciona una solución escalable y asequible, para aplicaciones pequeñas debido a las limitaciones de la cuenta. Además de que nos permite centrarnos en el desarrollo y la mejora de la aplicación sin preocuparnos por la infraestructura subyacente.



**¡Nos vemos
En la próxima
clase!**

